Web Architectures

Rimbert Rudisch-Sommer
rimbert.rudisch-sommer@fh-hagenberg.at

Recap: Strict Mode

- Strict Mode
  - changes "bad syntax" into real errors
    - results in more secure code
  - variables must be declared
    - variable names cannot be mistyped unnoticed any more
  - duplicating a function parameter is not allowed
  - duplicating an object property is not allowed
  - ...

- Declaration: "use strict";
  - at the beginning of the script: global scope
  - at the beginning of a function: local scope
  - Modules / Classes are automatically in strict mode

```
"use strict";
let x;
x = 1;
y = 2;  // This will cause an error
```

Recap: JavaScript Type System

- Types:
  - number
  - string
  - boolean
  - object
  - function
  - undefined

- operator "typeof"
  - typeof 13 => number
  - typeof "abc" => string
  - typeof [1,2,3] => object

Recap: JavaScript Type System

- null vs. undefined
  - null == undefined -> true
- Automatic Type Convertions
  - numerical operations
    - if operator is not + or no operand is a string
    - null -> 0, " " -> 0, false -> 0, true -> 1, NaN, ...
  - string concatenation (+)
    - null -> "null", undefined -> "undefined", true -> "true", false -> "false", ...
  - comparison ("==" vs "===" / "!=" vs "!==")
    - to number, if different types
    - null -> NaN (always false, if compared)
  - context (if (…))
    - 0, NaN -> false; "" -> false, " " -> true; null, undefined -> false

Recap: Classes

- Easier Syntax, but still prototype based: **`class <name> {...}`**
  - contains declaration of constructor, properties, getter/setter and other methods
- everything is public
  - exception: # as first character of name indicates private members
- class names not "hoisted" (must be declared before usage)

```
class A {
  #mx
  constructor() { this.#mx = 0; }
  get x() { return this.#mx; }
  set x(val) { this.#mx = val }
  hello() { return "my value is " + x; }
}

let a = new A();
console.log(a.x); // 0
a.x = 42;
console.log(a.x); // 42
console.log(a.hello()); // "my value is 42"
```

Recap: Classes

- inheritance works "as usual"

```
class B extends A {
  constructor() { super(); this.#mx = -1; }
  hello() { return "hello, my value is " + x; }
}
```

- static methods are executed on the class, no obj needed

```
class Util {
  static inc(x) { return x+1; }
  static dec(x) { return x-1; }
}

console.log(Util.inc(3));
```

Recap: Objects

- Objects contain
  - properties (values)
    - Can reference nested objects again
  - methods (functions)

- Accessible via dot-notation
  - <object/variable name>.<property/method name>

```
document.write("Hello World");
```

- Creating objects:
  - Assigning literal

```
let p= {x:5, y:10};
```

  - Constructior Function / Class + "new"

```
let d= new Date(); // creates new date object with current datestamp
```

Recap: Constructor Functions

- when a function is called together with the operator *new*, its *this* variable will reference a *new object*, that is *returned automatically*

```
function Rectangle(w, h) {

// properties
  this.width = w;
  this.height = h;

// method
  this.calcArea = function Rectangle_calcArea() {
    return this.width*this.height;
  }
}

// Usage:
let rect = new Rectangle(10,7);
rect.width= 5;
rect.calcArea();
```

Recap: Predefined Objects

- Math, Date, Number
- String

```
let s1= new String();
let s2= "This is a String";// string literal also creates object
s2.length -> 16
```

- Arrays

```
let a1= new Array(4,17,"MTD",3.15);
let a2= [2,3,5,7,11]  // construction via literal (cf. Strings)
Let v= a1[0]; a2[0]= 23;
```

  - length (last index+1), toString(), join(), reverse(), sort(), concat(), slice() …

- Associative Arrays

```
let color= new Array(),
color["red"]= "#FF0000";
let r= color.red;  // access via dot notation or [] notation
```

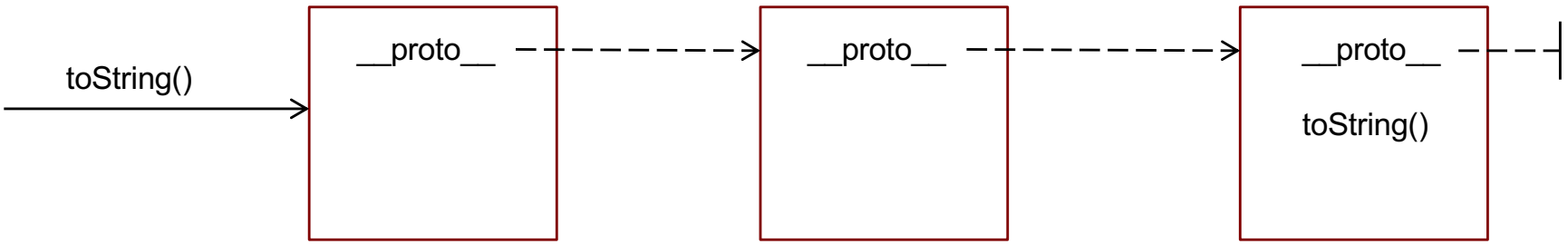
Recap: JavaScript: Prototypes

- JavaScript originally did not support classes
- but implemented the idea of inheritance via prototypes

- every object refers to a prototype object
  - this reference is a special property,
    standardwise read-/writeable with Object.getPrototypeOf()/Object.setPrototypeOf()
  - "most" implementations (browsers but IE < 11) provide "__proto__" (deprecated)

- the prototype object itself refers again to its prototype or null
  - "prototype chain"

- property / method lookup mechanism
  - if a property or method is accessed but not found in an object, the prototype is searched for it,
  - if not found in the prototype, it's prototype is searched ... until null (error)



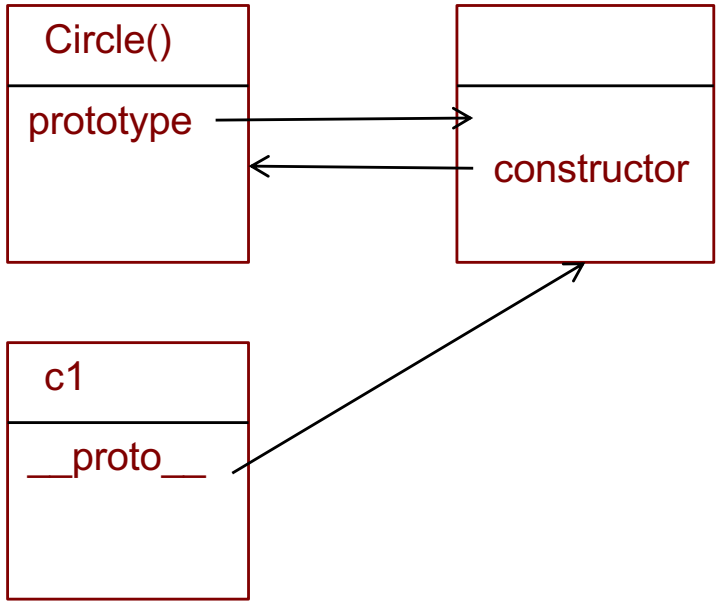
Recap: JavaScript: Prototypes

- Objects can be created via constructor functions + "new"
- Every function (is like an object) has a standard property "prototype" referencing an object
- This prototype object has a property "constructor" referencing back to the function object
- This prototype object is used as prototype (referenced by __proto__) for every object created when the function is called with "new"

```
function Circle(radius) {
    ...
}

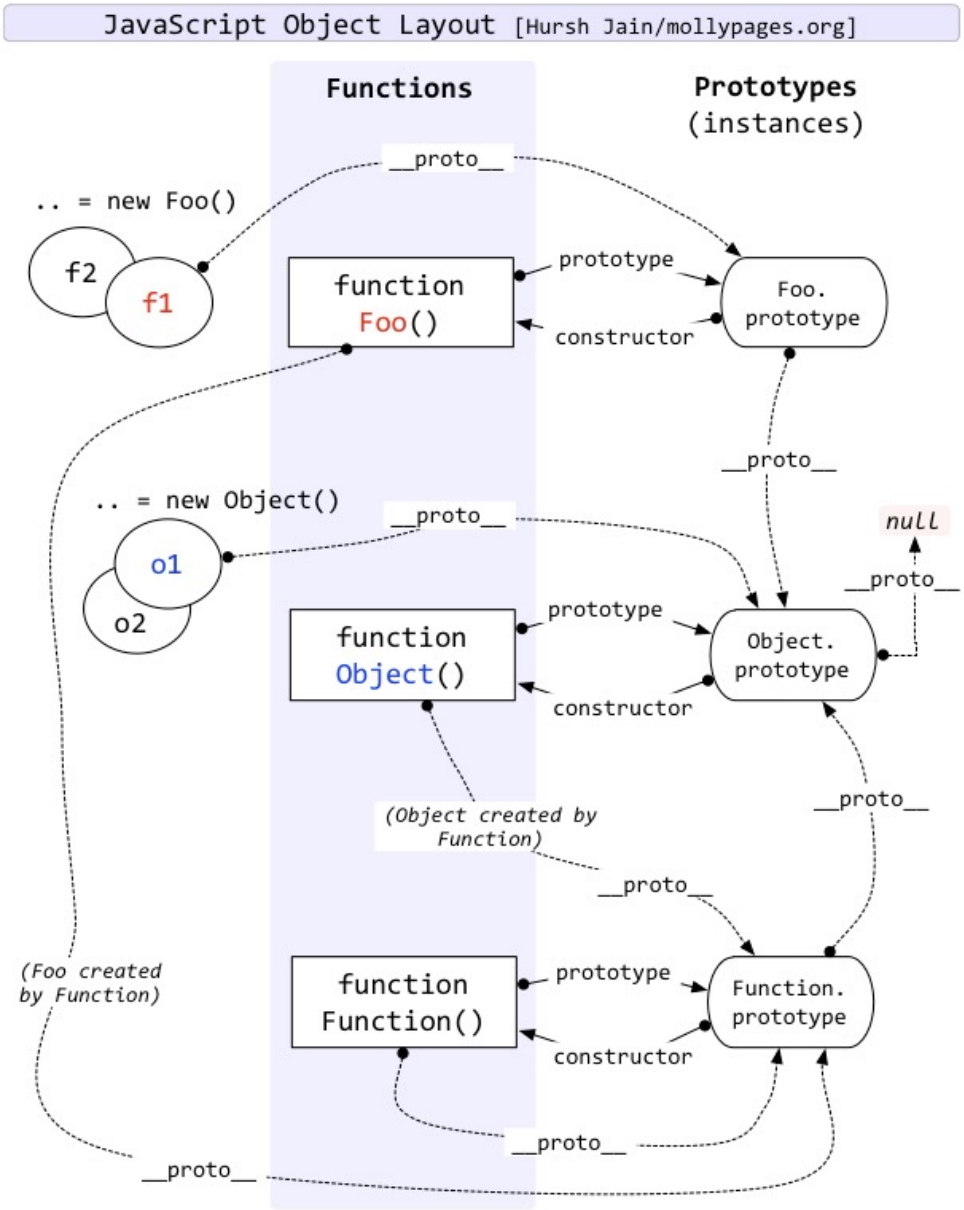
let c1= new Circle(5);

// c1.constructor => Circle()
```



Recap: JavaScript: Prototypes

- Functions (are objects) and therefore also have a __proto__ property!!
- also the Object() function (constructor for {})
- also the Function() function (used to create function objects)



Recap: Mixins

- JavaScript does not allow multiple inheritance
- but the possibility of adding code to prototype
  - **`Object.assign(target, source)`**
    copies all source properties to target

```
let helloMixin = {
  sayHello() { alert(`Hello, my name is ${this.name}.`); }
}

class User {
  constructor(name) { this.name = name; }
}

Object.assign(User.prototype, helloMixin);

new User("Bert").sayHello();
```

Recap: Same Origin Policy (SOP)

- browser regulation - introduced for security reasons:
- a script may access only resources from it's own origin

- Origin = Protocol + Host + Port

  http://www.example.com/ != **https**://www.example.com/
  http://www.**example.com**/ != http://www.**nasa.gov**/
  http://www.example.com/ != http://www.example.com**:8080**/

- HTML5 Storage APIs:
  data is bound to documents of same origin
  - cannot access data from other origins

Recap: Web Storage API

- localStorage and sessionStorage provide the same API
  - functions or dot-notation:
    - setItem(key, data) …Storage.key= data;
    - getItem(key) …Storage.key;
      - returns null, if key non-existent
    - removeItem(key) delete …Storage.key;
  - additional properties (methods)
    - length number of entries
    - key(index) key of x.th entry
    - clear() clear all
  - Example: read all entries

```
for (var i=0; i<sessionStorage.length; i++) {
  var key= sessionStorage.key(i);
  alert(key+sessionStorage.getItem(key));
}
```

Recap: Web Storage + JSON

- to store objects
- JSON API:
  - JSON.stringify — json -> String
    - calls the objects toJSON() method
  - JSON.parse — String -> json

- Example
  - store an object

```
var  person = {
  name: 'Herman Munster',
  age: '33'
}
sessionStorage.setItem('person',JSON.stringify(person));
```

  - read object

```
var person= JSON.parse(sessionStorage.getItem('person'));
// JSON.parse(null) results to null
```

Recap: Web Storage Events

- if data is manipulated in one window, all other windows affected are notified by events

- StorageEvent-object
  - key
  - oldValue
  - newValue
  - url
  - storageArea

- Example

```
function handleStorage(event) {
  alert("Value of "+event.key+ "changed to: "+event.newValue);
}
window.addEventListener('storage', handleStorage, false);
```

Modules

- import, export keywords
- to get access to a module feature it must be explicitly "exported"

```
export const name = 'WAR3';
export function draw() {...};
```

- or single export statement

```
export {name, draw};
```

- to use those features, other scripts can "import" them

```
import {name, draw} from './mymodule.js';
console.log(name);
draw();
```

- to load the module via script tag use attribute **`type="module"`**
  - Same Origin Policy is applied!

```
<script type="module" src="mymodule.js"></script>
```

Modules

- default exports
  - if only one value is exported, it can be made the default export
  - no extra name needed
  - eg. file "uppercase.js"

```
export default str => str.toUpperCase()
```

- Syntax for import:

```
import toUpperCase from './uppercase.js' // {} optional if one name

toUpperCase('test'); // 'TEST'
```

- general import syntax: * imports everything

```
import * from ...
```

JavaScript Functions

- Functions can be used as value, anonymous function expression

```
let a = function() { ... }
```

```
let a = () => {}
```

- Functions as value, named function expression
  - name available only from within the function, useful eg. for recursion

```
let a = function a() { ... }
```

- Self-Evaluating functions
  - aka. IEFE = immediately executed function expression

```
let a = (function() {return 1;})() ==> a = 1
```

- Inner functions are allowed

```
function foo () {
  function bar() {
  }
}
```

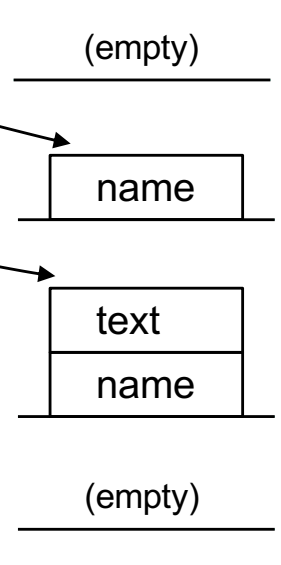
Functions: local Scope

- Lexical (static) Scoping
  - local name scope (let / var / const)
- the JavaScript engine manages an internal datastructure for storing local variables
- local variables of a function are
  - created on the stack when they are declared
  - removed from the stack when the function ends

```
function greeting(name) {

  let text = "Hello " + name + "!";

  return text;
}
```



Closures

- special programming concept concerning "inner" functions and their access of variables
  - usually local variables of a function are removed when the function ends

- Javascript treats functions as objects
  - functions can be assigned to variables

```
let a= function() {alert('hello')};
a(); // call the function via variable
```

  - functions can be passed as parameters

```
button.addEventListener('click', function(event) {...})
```

  - functions can be used as return values

```
function test1() {
   return function() {alert('hello');} // anonymous inner function!
}
let b= test1();
b(); // call the returned function via variable
```

- A closure is created when
  - an inner function references local variables of the outer function
  - while the outer function has ended.

Closures: Example

- if a function is used as return value, it keeps the variables of the outer function if needed

```
function makeClosure() {
  let text="Hello ";
  return function(name) {alert(text+name);} // anonymous inner function
}

let simpleHello= makeClosure();  // outer function ends,
                                 // variable text kept in a closure

simpleHello("World!");  // calls returned inner function
                        // Output: "Hello World!"
```

- this can be used to parameterize the returned function:

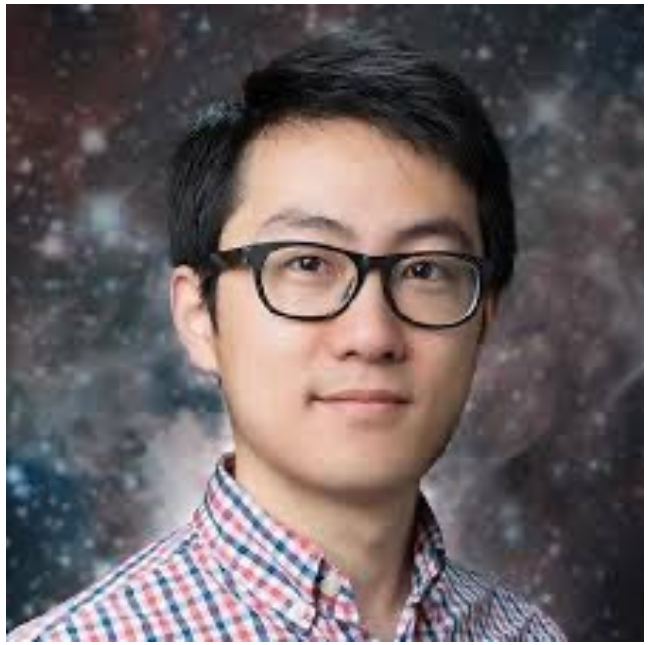
```
function createParamGreeting(greeting) {      // parameter has same scope as local var
  return function(name) {alert(greeting + name);}
}

let helloGreeting= createParamGreeting("Hello ");
helloGreeting("World!");  // alerts "Hello World!"

let howdyGreeting= createParamGreeting("Howdy ");
howdyGreeting("again"); // alerts "Howdy again!"
```

Vue.js ("view")

- Progressive JS framework for building user interfaces
  - Incrementally adoptable
- Created by Evan You, first released Feb 2014
- http://viewjs.org/
- Declarative rendering (templates)
- Easy event binding
- "Reactive" two-way binding
  - Model <-(updates)-> UI Element
- Component-based
- Incremental



Installation / APIs

- in the most simple scenario just includes one javascript file (locally or CDN)

- more complex development workflows are provided including using different tools (see later in the semester)
  - vite

- Two APIs how to access the library:
  - Options API (came first, not tools needed)
  - Composition API (structural advantages, tools needed)

What does a typical JavaScript application do?

```
[…]
let notesData = [];

function renderData(){
    let output = "";
    for(let entry of notesData){
        output += `<em>${entry.date}</em> -- ${entry.content}<br>`;
    }
    document.querySelector("#notes").innerHTML = output;
}

function saveData() { […] renderData();}
function loadData() { […] renderData();}

document.querySelector("#save").addEventListener('click', saveData);
loadData();
```

manage some data

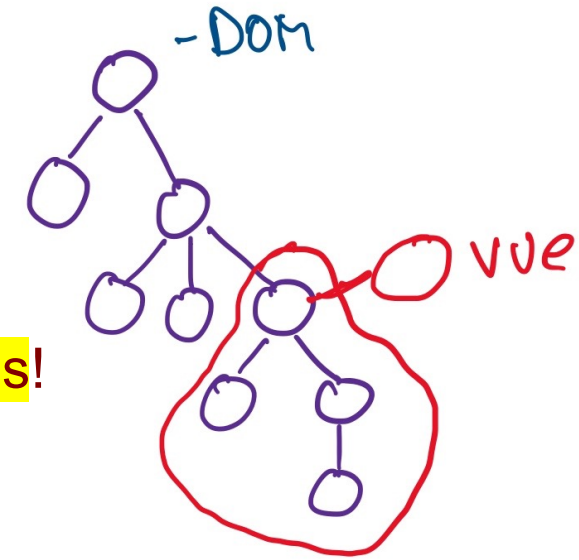
render the data on the webpage

change the data on certain events, eg. user input (and re-render)

Vue Basic Idea

- Vue lets you create instances of itself

- a Vue instance
  - is attached to a node of the DOM tree (HTML element)
  - manages data
  - renders the data in the subtree of the node
    - declaratively via templates
  - automatically re-renders when the data changes!
  - manages functions (event handlers)
  - receives events occuring in the subtree of the attached node
    - to be able to manipulate the data



Vue Instance & Declarative Rendering

- A Vue Instance is created via **`Vue.createApp()`**
  - takes options object as parameter (Options API)
  - "mounted" at a DOM element

```
Vue.createApp({
  data() {
    return { message: 'Hello WAR3!' };
  }
}).mount('#app');
```
JavaScript

data – function returning "variables" of the instance,

mounted at element '#app'

- declaratively render data to the DOM using *templates*:

```
<div id="app">
  {{ message }}
</div>
```
HTML

- every change of the instance data also updates DOM automatically (*reactivity system*)
  - Here: if **`message`** gets changed in JavaScript, the value changes on the web page also automatically

Templates

- Plain text: "mustache syntax"
  - meta characters (<,>, …) are escaped (< , > , …)

```
{{ message }}
```

- Raw HTML: *directive v-html*
  - does not escape meta characters (<,>,…)

```
<span v-html="message"></span>
```

- Single JavaScript expressions are allowed

```
<span>{{ message + " again"}}</span>
```

Template Directives

- Special attributes with prefix "v-"
- Values must be *single JavaScript expressions*
- Arguments denoted by colon :

- v-bind: Bind attribute values reactively

```
<div id="app">
  <span v-bind:title="message">Vue.js</span>
</div>
```

- v-on: Set event handler (example see next slide)

- Shortcuts
  - v-bind:title – *:title*
  - v-on:click – *@click*

User Input / Event Handling

- Define functions with Vue instance option *methods* and attach them as event handlers with directive *v-on*:

```
Vue.createApp({
  data() {
    return {message: 'Hello WAR3!'};
  },
  methods: {
    plusMessage() {
      this.message = this.message+' again';
    }
  }
}).mount('#app');
```

```
<div id="app">
  <p>{{ message }}</p>
  <button v-on:click="plusMessage">Enhance Message</button>
</div>
```

Shortcut: `@click="plusMessage"`

Vue Data

- *on creation* of the Vue instance, all elements of its *data property* are added to *reactivity system*
  - automatic DOM updates
  - elements added later (after initial creation) will not be reactive

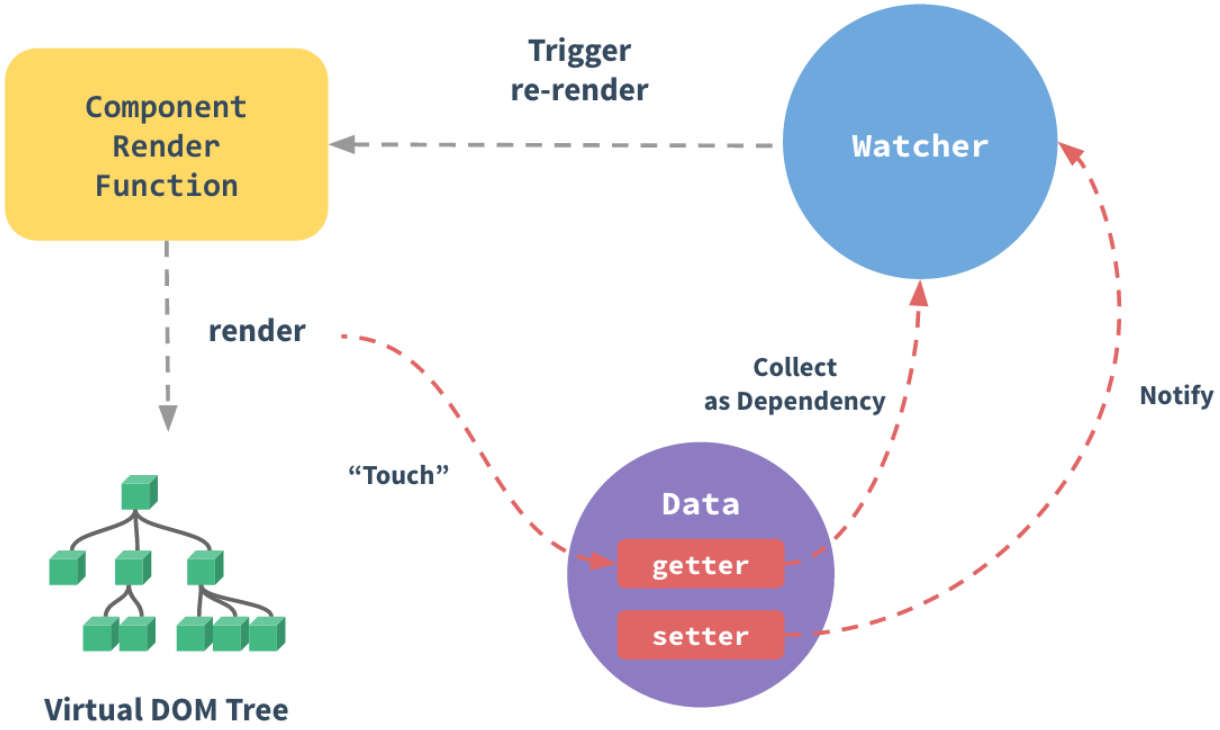
- user-defined properties are accessible via dot-notation .
- vue "library" properties are accessible via prefix $

```
let app = Vue.createApp({
  data() {
    return { message: 'Hello WAR3 students!' };
  }
}).mount('#app');

app.message = "New Value";
app.$el // readonly, DOM Object with ID '#app'
```

Implementation of Reactivity

- On initialization Vue iterates over all data properties
- Converts them to getter/setter methods / creates proxy objects
- Invisible to the user dependency tracking is enabled
- Every instance has a "watcher" recording properties used in rendering
- When one of the setter methods is called triggers re-render



Directives: Conditional Rendering

- v-if

```
<h3 v-if="ok">Yes</h3>                    // ok is boolean (true/false)
```

- v-else (must follow immediately after v-if)

```
<h3 v-if="ok">Yes</h3>
<h3 v-else>No</h3>
```

- v-else-if

```
<h3 v-if="type === 'A'">Type A</h3>
<h3 v-else-if="type === 'B'">Type B</h3>
<h3 v-else>Not Type A or B</h3>
```

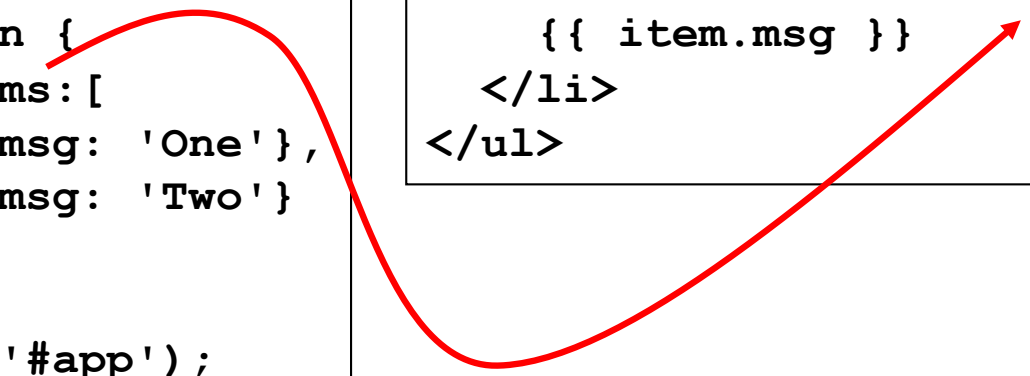
- v-show
  - like v-if, but DOM element is always rendered but possibly hidden (display:none)

Directives: List Rendering

- v-for
  - Special value syntax: *item in items*

```
Vue.createApp({
  data() {
    return {
      items:[
        {msg: 'One'},
        {msg: 'Two'}
      ]
    }}
}).mount('#app');
```

```
<ul id="app">
  <li v-for="item in items">
    {{ item.msg }}
  </li>
</ul>
```



  - Use

```
v-for="(item, index) in items"
```

    if the indizes of the elements are needed (names of loop variables can be chosen individually)

  - Vue also wraps the following array's mutation methods for reactiveness: push(), pop(), shift(), unshift(), splice(), sort(), reverse()

Directives: Form Input

- v-model
  - "two-way binding"



```
Vue.createApp({
  data() {
   return { message: '' };
  }
}).mount('#app');
```

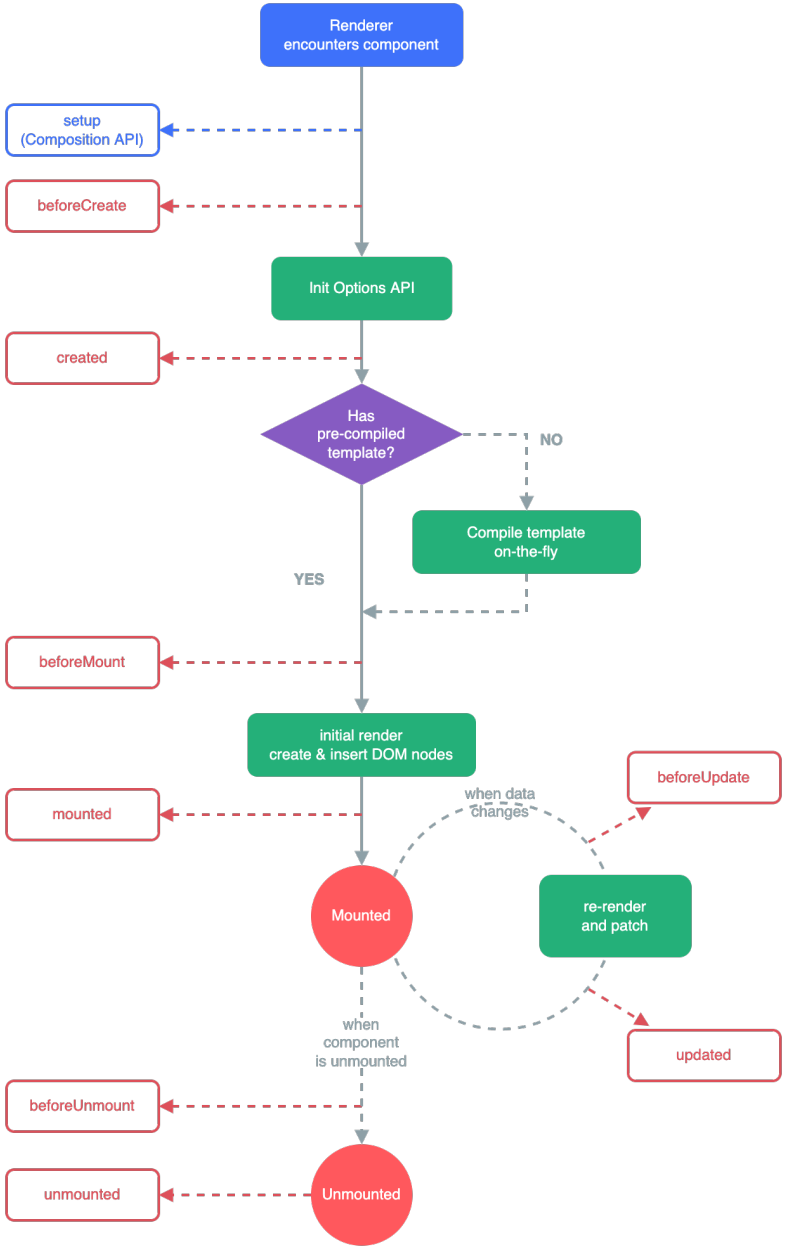
```
<div id="app">
  <input type="text" v-model="message">
  <p>Message is: {{ message }}</p>
</div>
```

- Also for textarea, checkbox, radiobuttons, select, …

Vue Instance Lifecycle Hooks

- Methods with a given name
  - called automatically at certain points in the lifetime of a Vue instance
- beforeCreate(), created()
  - before/after setting up reactivity system
  - only called once
- beforeMount(), mounted()
  - before/after setting up and syncing virtual dom (el gets connected)
  - maybe called multiple times
- beforeUpdate(), updated()
  - before/after re-rendering updated values
- beforeUnmount(), unmounted()
  - before/after tearing down subelements/reactivity system

Vue Lifecycle



Computed Properties

- Vue option "computed"
- to compute a value from existing data
  - the function is used as getter function
  - value gets cached, only changes when dependencies change

```
Vue.createApp({
  data() {
    return {
      firstName: '…',
      lastName: '…'
  }},
  computed: {
    fullName() {
      return this.firstName + " " + this.lastName;
  }}}).mount('#app');
```

```
<div id="app">
  <p>{{ fullName }}</p>
</div>
```

Class / Style Bindings

- v-bind:class, v-bind:style
  - Object Syntax: toggle styles

```
data(){ return {
  isActive: true,
}}
```

```
<div class="static"
   v-bind:class="{ active: isActive }" >
</div>
```

  - Array Syntax: apply styles

```
data(){ return {
  myClass: 'flashy',
}}
```

```
<div class="static"
   v-bind:class="[myClass]">
</div>
```

  - mixing object and array style

```
<div v-bind:class="[{ active: isActive }, myClass]" >
</div>
```

Vue Data again

- on creation of the Vue instance, all properties of its data object are added to *reactivity system*
- user-defined properties accessible via dot-notation, eg. this.message

- properties added later will not be part of the reactivity system
  - eg in created() or mounted() functions

```
Vue.createApp({
  data() {return {}},
  methods: {
    // can access this.tmpVar …
  },
  mounted: function() {
    this.tmpVar = "Not reactive";
  }
}).mount("#app");
```

Virtual DOM

- it's "expensive" to update the DOM
  - node lookup (eg by id) in thousands of nodes
  - re-render after change

- a lot of small, frequent changes will slow the page down

- create a data structure similar to dom (aka. virtual dom)
  - representing the parts of the page we want to change
  - changing the virtual dom is less expensive
  - only sync with real dom at certain times -> less re-renderung

```
let domNode = {
  tag: 'ul',
  attributes: { id: 'myId' },
  children: [ // where the LI's would go ]
};
```

Vue: $refs, $nextTick()

- $refs
  - to access html elements from Vue instance programmatically

```
<input ref="myInput" ...>

$refs.myInput.select();
$refs['myInput'].select();
```

- $nextTick(callback)
  - defer the callback to be executed after the next DOM update
    - use it after you've changed some data to wait for the DOM update

```
<div ref="msg">{{ message }}</div>          // data: {message: 'old'}

this.message = 'new';
console.log(this.$refs.msg.textContent);    // 'old'
this.$nextTick(function() {
  console.log(this.$refs.msg.textContent)}); // 'new'
```